

CSE 396 — Computer Engineering Project

Group 10

LIDAR-Guided Hospital Navigation Robot

Module Documentation — Assignment 3

Team Members

Name	Student ID	Module
Muhammet Gökhan ÖZBEK	210104004038	MOD-01 Voice Communication (Primary)
Samet ALKAN	230104004034	MOD-02 Digital Twin & Visualization (Primary)
Hicran KOÇ	220104004074	MOD-03 Hardware & Motor Control (Primary)
Fatih Emre OĞAN	230104004090	MOD-04 SLAM (Co-primary)
Mervan Deniz YILDIRIM	230104004173	MOD-04 SLAM (Co-primary)
Ertuğrul ERGÜL	240104004994	MOD-05 NAV Path Planning (Co-primary)
Ömer Faruk GÜRSEL	210104004053	MOD-05 NAV Path Planning (Co-primary)
Sayed Khalilullah HASHİMİ	200104004806	MOD-06 Sensor Integration (Primary)

1. System Architecture Overview

The LIDAR-Guided Hospital Navigation Robot is a differential-drive mobile robot that autonomously guides visitors to their destinations within a hospital floor. All computation runs on a Raspberry Pi 4 (2 GB), which hosts six cooperating software modules. These modules exchange structured data through a shared type library (`project_types.h`) and communicate via in-process queues and callbacks.

1.1 Module Summary

Module ID	Name	Primary Owner	One-sentence Role
MOD-01	Voice Communication	Gökhan ÖZBEK	Converts spoken Turkish commands into <code>nav_goal_t</code> and speaks <code>nav_status_t</code> back to the user.
MOD-02	Digital Twin & Viz.	Samet ALKAN	Streams robot telemetry over WebSocket and renders the robot state on a 3-D floor map.
MOD-03	Hardware & Motor	Hicran KOÇ	Owns all GPIO — PWM drive, encoder reading, IMU, and battery monitoring.
MOD-04	SLAM	Fatih / Mervan	Builds and maintains the 2-D occupancy grid and publishes the real-time pose estimate.
MOD-05	NAV – Path Planning	Ertuğrul / Ömer	Plans collision-free paths and drives the robot to waypoints via a proportional heading controller.
MOD-06	Sensor Integration	Sayed HASHİMİ	Fuses LiDAR forward arc and proximity sensors to produce consolidated obstacle alerts.

1.2 Data Flow Between Modules

The diagram below summarises every inter-module data exchange. Arrows show the direction of data production.

From	To	Data Type	Rate	Notes
MOD-01	MOD-05	<code>nav_goal_t</code>	On voice command	Destination name + <code>waypoint_id</code> from STT
MOD-05	MOD-01	<code>nav_status_t</code>	On FSM state change	Turkish TTS message for the speaker
MOD-03	MOD-04	<code>odometry_data_t</code>	50 Hz	Wheel tick counts and computed speeds
MOD-03	MOD-04	<code>imu_data_t</code>	100 Hz	Gyro yaw-rate, accelerometer XYZ, temperature
MOD-05	MOD-03	<code>motion_cmd_t</code>	50 Hz	Left/right PWM duty cycles + emergency stop flag
MOD-03	MOD-05	<code>motion_status_t</code>	50 Hz	Actual wheel speeds and stall detection flag
MOD-04	MOD-05	<code>slam_state_t</code>	20 Hz	Robot pose (x, y, θ) and ICP confidence
MOD-04	MOD-05	<code>occ_map_t</code>	On map update	Pointer to 2-D occupancy grid (5 cm/cell)
MOD-05	MOD-04	<code>checkpoint_req_t</code>	On junction arrival	Pose-snap request at corridor checkpoint
MOD-04	MOD-05	<code>checkpoint_ack_t</code>	On checkpoint response	Snapped pose or failure flag
MOD-06	MOD-05	<code>obstacle_alert_t</code>	20 Hz	Active flag, min distance, bearing to obstacle
MOD-06	MOD-04	<code>sensor_reading_t</code>	10 Hz	Raw ultrasonic/IR readings for map enrichment
MOD-04	MOD-02	<code>telemetry_t</code>	5–10 Hz	Pose + nav status + battery, WebSocket JSON

From	To	Data Type	Rate	Notes
MOD-05	MOD-02	nav_status_t	On FSM state change	Navigation overlay for Digital Twin
MOD-06	MOD-02	sensor_reading_t	5 Hz	Raw sensor snapshot for visualization

1.3 Shared Type Library (project_types.h)

All types that cross module boundaries are declared in a single shared header at the repository root. No inter-module struct is defined inside a module header. The table below lists every shared type, its location in the codebase, and which modules use it.

Struct / Enum	Defined in	Producer	Consumer(s)	Purpose
nav_goal_t	project_types.h	MOD-01	MOD-05	Spoken destination → waypoint mapping
nav_state_e	project_types.h	MOD-05	MOD-01, MOD-02	FSM states (IDLE/MOVING/WAITING/...)
nav_status_t	project_types.h	MOD-05	MOD-01, MOD-02	State + Turkish TTS message
odometry_data_t	project_types.h	MOD-03	MOD-04	Encoder ticks and wheel speeds
imu_data_t	project_types.h	MOD-03	MOD-04	Gyroscope and accelerometer data
slam_state_t	project_types.h	MOD-04	MOD-05, MOD-02	Robot pose and ICP confidence
occ_map_t	project_types.h	MOD-04	MOD-05	2-D occupancy grid pointer + metadata
motion_cmd_t	project_types.h	MOD-05	MOD-03	PWM commands for left/right motors
motion_status_t	project_types.h	MOD-03	MOD-05	Actual speeds and stall flag
sensor_reading_t	project_types.h	MOD-06	MOD-04, MOD-02	Raw ultrasonic/IR distance reading
obstacle_alert_t	project_types.h	MOD-06	MOD-05	Consolidated obstacle detection result
checkpoint_req_t	project_types.h	MOD-05	MOD-04	Pose-snap request at junction
checkpoint_ack_t	project_types.h	MOD-04	MOD-05	Snapped pose result / failure
telemetry_t	project_types.h	MOD-04/05	MOD-02	Combined pose + nav + battery snapshot

2. MOD-01 — Voice Communication

2.1 What This Module Does

MOD-01 is the robot's spoken interface. It continuously listens through a USB microphone (Youmi Mini USB 2.0), transcribes the audio offline using a Turkish speech-to-text engine (Whisper tiny or Vosk), maps recognised phrases to destination waypoints, and forwards a `nav_goal_t` to MOD-05. In the opposite direction, it receives `nav_status_t` state updates from MOD-05 and reads them aloud through the speaker (MakerHawk Mini Speaker + DROK PAM8406 amplifier) using Piper TTS (`tr_TR-dfki-medium`).

The module runs entirely on-device — no cloud or network connectivity is required — making it resilient to hospital Wi-Fi outages.

2.2 Public API

Function	Signature	Returns	Description
<code>voice_init</code>	<code>voice_init(void)</code>	<code>voice_status_t</code>	Initialises microphone and TTS engine. Call once at startup.
<code>voice_start_listen</code>	<code>voice_start_listen(void)</code>	<code>voice_status_t</code>	Begins continuous STT listening loop.
<code>voice_stop_listen</code>	<code>voice_stop_listen(void)</code>	<code>voice_status_t</code>	Stops listening and releases audio input resources.
<code>voice_speak</code>	<code>voice_speak(const char *text)</code>	<code>voice_status_t</code>	Synthesises and plays Turkish speech from a UTF-8 string.
<code>voice_get_goal</code>	<code>voice_get_goal(nav_goal_t *out)</code>	<code>voice_status_t</code>	Blocks until a spoken command is parsed into a <code>nav_goal_t</code> .
<code>voice_set_status_cb</code>	<code>voice_set_status_cb(voice_status_cb_t cb)</code>	<code>void</code>	Registers or clears the callback for <code>nav_status_t</code> updates.

2.3 Key Constants

Constant	Value	Meaning
<code>VOICE_MAX_CMD_LEN</code>	64	Maximum recognised command length (characters)
<code>VOICE_SPEAK_TIMEOUT_MS</code>	5 000	Maximum TTS synthesis and playback wait (ms)
<code>VOICE_VERSION</code>	"0.1"	Interface version string

2.4 Connections to Other Modules

Direction	Partner Module	Data Type	Rate	Transport
MOD-01 → MOD-05	NAV Path Planning	<code>nav_goal_t</code>	On voice command	In-process queue / callback
MOD-05 → MOD-01	NAV Path Planning	<code>nav_status_t</code>	On FSM state change	<code>voice_status_cb_t</code> callback

2.5 Return Codes

Code	Value	Meaning
VOICE_OK	0	Success
VOICE_ERR_MIC	-1	Microphone initialisation failed
VOICE_ERR_STT	-2	Speech-to-text engine error
VOICE_ERR_TTS	-3	TTS synthesis or playback error
VOICE_ERR_TIMEOUT	-4	No command recognised within the timeout window

2.6 External Dependencies

Library / Tool	Purpose
Whisper tiny / Vosk	Offline Turkish speech-to-text
Piper TTS (tr_TR-dfki-medium)	Offline Turkish text-to-speech
ALSA / PyAudio	Linux audio backend for mic capture and speaker output
project_types.h	Shared nav_goal_t and nav_status_t definitions

3. MOD-02 — Digital Twin & Visualization

3.1 What This Module Does

MOD-02 creates a real-time 3-D mirror of the robot. It runs a WebSocket server (default port 8765) that accepts connections from a Unity 2022 LTS or Gazebo renderer and from a browser-based dashboard. At 10 Hz it serialises a `telemetry_t` snapshot (pose, navigation state, battery voltage, uptime) into JSON and broadcasts it to all connected clients. Operators can also place coloured priority pins on the floor map to mark detected points of interest, and clear them when no longer relevant.

MOD-02 is a pure consumer — it does not issue commands to any other module.

3.2 Public API

Function	Signature	Returns	Description
<code>dtwin_init</code>	<code>dtwin_init(const char *map_path, uint16_t port)</code>	<code>dtwin_status_t</code>	Initialises WebSocket server and loads the floor-map file. Must be called once before any other function.
<code>dtwin_push_telemetry</code>	<code>dtwin_push_telemetry(const telemetry_t *t)</code>	<code>dtwin_status_t</code>	Serialises and broadcasts one telemetry frame to all connected clients at 10 Hz.
<code>dtwin_drop_pin</code>	<code>dtwin_drop_pin(float x_m, float y_m, uint8_t priority)</code>	<code>dtwin_status_t</code>	Drops a priority pin (0=low ... 3=critical) at the given map coordinates.
<code>dtwin_clear_pins</code>	<code>dtwin_clear_pins(void)</code>	<code>dtwin_status_t</code>	Removes all active pins from the map overlay and resets the pin counter.
<code>dtwin_shutdown</code>	<code>dtwin_shutdown(void)</code>	<code>void</code>	Sends a graceful WebSocket close, joins the listener thread, and frees the map asset.

3.3 Key Constants

Constant	Value	Meaning
<code>DTWIN_PORT_DEFAULT</code>	8765	Default WebSocket listening port
<code>DTWIN_PIN_MAX</code>	32	Maximum simultaneous map pins
<code>DTWIN_TELEMETRY_HZ</code>	10	Expected telemetry push rate (Hz)
<code>DTWIN_INTERP_WINDOW_MS</code>	200	Dead-reckoning interpolation window when feed lags (ms)
<code>DTWIN_VERSION</code>	"0.1"	Interface version string

3.4 Connections to Other Modules

Direction	Partner	Data Type	Rate	Transport
MOD-04 → MOD-02	SLAM	<code>telemetry_t</code> (pose part)	5 Hz	TCP WebSocket JSON
MOD-05 → MOD-02	NAV	<code>nav_status_t</code> (inside <code>telemetry_t</code>)	On state change	TCP WebSocket JSON
MOD-06 → MOD-02	Sensors	<code>sensor_reading_t</code> snapshot	5 Hz	TCP WebSocket JSON

3.5 Return Codes

Code	Value	Meaning
DTWIN_OK	0	Success
DTWIN_ERR_CONN	-1	WebSocket connection unavailable or lost
DTWIN_ERR_ENCODE	-2	JSON serialisation failed
DTWIN_ERR_MAP	-3	Floor-map file could not be loaded or parsed
DTWIN_ERR_PIN	-4	Pin limit exceeded (DTWIN_PIN_MAX = 32)

4. MOD-03 — Hardware & Motor Control

4.1 What This Module Does

MOD-03 owns every low-level hardware interaction on the Raspberry Pi 4. It generates 50 Hz PWM signals for the L298N H-bridge motor driver to spin the two DC geared motors, counts quadrature encoder pulses via GPIO edge-detect interrupts to compute wheel odometry, reads the MPU-6050 IMU over I²C at 400 kHz to obtain gyroscope and accelerometer data, and monitors the LiPo battery voltage through a resistor divider on an ADC pin.

No other module accesses GPIO or I²C directly — they call MOD-03 functions exclusively. MOD-03 exposes a motion API (forward / backward / turn / stop) to MOD-05 and publishes odometry and IMU data to MOD-04.

4.2 Hardware Connections

Signal	RPi BCM Pin	Device
L298N IN1 / IN2	GPIO 5, 6	Left motor forward / reverse
L298N ENA	GPIO 12 (PWM0)	Left motor speed (50 Hz PWM)
L298N IN3 / IN4	GPIO 13, 19	Right motor forward / reverse
L298N ENB	GPIO 26 (PWM1)	Right motor speed (50 Hz PWM)
Encoder Left A / B	GPIO 17, 18	Left wheel quadrature channels
Encoder Right A / B	GPIO 22, 23	Right wheel quadrature channels
MPU-6050 SDA / SCL	GPIO 2, 3	I ² C @ 400 kHz (addr 0x68)

4.3 Public API

Function	Signature	Returns	Description
hw_init	hw_init(void)	hw_status_t	Initialises GPIO, PWM, encoder ISR, I ² C, and MPU-6050 registers.
hw_set_motion	hw_set_motion(const motion_cmd_t *cmd)	hw_status_t	Applies left/right PWM targets from MOD-05. Clamps to ± 0.5 m/s. Sets cmd→stop triggers emergency stop.
hw_emergency_stop	hw_emergency_stop(void)	hw_status_t	Immediately cuts both motors. Safe from any thread or signal handler.
hw_get_odometry	hw_get_odometry(odometry_data_t *out)	hw_status_t	Atomically snapshots encoder delta counters and computes wheel speeds. Call at 50 Hz.
hw_get_imu	hw_get_imu(imu_data_t *out)	hw_status_t	Synchronous I ² C burst read of gyroscope and accelerometer registers. Call at 100 Hz.
hw_get_battery_voltage	hw_get_battery_voltage(float *v_out)	hw_status_t	ADC read + calibration → battery voltage in volts. Readings < 10.5 V indicate low battery.

4.4 Key Constants

Constant	Value	Meaning
HW_ENCODER_CPR	20	Encoder counts per wheel revolution
HW_WHEEL_DIAMETER_M	0.065 m	Wheel diameter
HW_WHEELBASE_M	0.17 m	Track width between wheel centres
HW_PWM_FREQ_HZ	50 Hz	L298N PWM carrier frequency
HW_MAX_SPEED_MPS	0.5 m/s	Safety cap — speeds clamped before PWM write
HW_IMU_I2C_ADDR	0x68	MPU-6050 I ² C address (AD0 = GND)
HW_IMU_COMP_ALPHA	0.98	Complementary filter blending coefficient

4.5 Connections to Other Modules

Direction	Partner	Data Type	Rate	Notes
MOD-03 → MOD-04	SLAM	odometry_data_t	50 Hz	Left/right tick counts and speeds
MOD-03 → MOD-04	SLAM	imu_data_t	100 Hz	Gyro yaw-rate, accel XYZ, temperature
MOD-05 → MOD-03	NAV	motion_cmd_t	50 Hz	PWM targets + emergency stop flag
MOD-03 → MOD-05	NAV	motion_status_t	50 Hz	Actual speeds and stall flag

4.6 Return Codes

Code	Value	Meaning
HW_OK	0	Success
HW_ERR_GPIO	-1	GPIO initialisation or write failure
HW_ERR_I2C	-2	I ² C bus error (MPU-6050 NACK or bus hang)
HW_ERR_TIMEOUT	-3	Operation missed its real-time deadline

5. MOD-04 — SLAM (Mapping & Localization)

5.1 What This Module Does

MOD-04 implements Simultaneous Localization and Mapping using the KISS-ICP LiDAR odometry pipeline. It reads 360° scans from the RPLidar A2 and fuses them with wheel odometry (`odometry_data_t` at 50 Hz) and IMU data (`imu_data_t` at 100 Hz) received from MOD-03. From this fusion it maintains a 2-D occupancy grid (5 cm per cell) of the hospital floor and a continuously updated pose estimate (x, y, θ) that it publishes to MOD-05 at 20 Hz.

At predefined corridor-junction checkpoints, MOD-04 performs scan-to-reference matching. If the ICP confidence meets the 0.80 threshold, it snaps the pose to the known checkpoint coordinates and acknowledges the request to MOD-05, resetting accumulated drift. The occupancy grid is also forwarded to MOD-02 inside a `telemetry_t` for visualization.

5.2 Public API

Function	Signature	Returns	Description
<code>slam_init</code>	<code>slam_init(const char *map_path)</code>	<code>slam_status_t</code>	Opens RPLidar A2, allocates the occupancy grid, optionally loads a pre-built .pgm/.yaml map.
<code>slam_update</code>	<code>slam_update(const odometry_data_t *odom, const imu_data_t *imu)</code>	<code>slam_status_t</code>	Feeds one control-loop tick of odometry and IMU data into KISS-ICP. Call at 50–100 Hz.
<code>slam_get_pose</code>	<code>slam_get_pose(slam_state_t *out)</code>	<code>slam_status_t</code>	Thread-safe copy of the current pose estimate (x, y, θ , confidence).
<code>slam_get_map</code>	<code>slam_get_map(uint8_t **grid_out, uint16_t *w, uint16_t *h)</code>	<code>slam_status_t</code>	Returns a read-only pointer to the current occupancy grid. Valid until next map-modifying update.
<code>slam_reset_pose</code>	<code>slam_reset_pose(float x_m, float y_m, float theta_rad)</code>	<code>slam_status_t</code>	Overrides the internal pose estimate (called by MOD-05 after a successful checkpoint snap).
<code>slam_process_checkpoint</code>	<code>slam_process_checkpoint(const checkpoint_req_t *req, checkpoint_ack_t *ack)</code>	<code>slam_status_t</code>	Runs scan-to-reference matching. Fills ack with CP_ACK_OK and snapped pose, or CP_ACK_FAIL.
<code>slam_shutdown</code>	<code>slam_shutdown(void)</code>	<code>void</code>	Stops RPLidar motor, frees grid memory, and joins internal threads.

5.3 Key Constants

Constant	Value	Meaning
<code>SLAM_GRID_RESOLUTION_M</code>	0.05 m	Occupancy grid cell size
<code>SLAM_CHECKPOINT_THRESH</code>	0.80	Minimum ICP confidence to accept a pose snap
<code>SLAM_LIDAR_MAX_RANGE_M</code>	8.0 m	RPLidar A2 usable range cap
<code>SLAM_UPDATE_HZ</code>	20 Hz	Pose publish rate to MOD-05
<code>SLAM_VERSION</code>	"0.1"	Interface version string

5.4 Connections to Other Modules

Direction	Partner	Data Type	Rate	Notes
MOD-03 → MOD-04	Hardware	odometry_data_t	50 Hz	Encoder tick counts and computed speeds
MOD-03 → MOD-04	Hardware	imu_data_t	100 Hz	Gyroscope and accelerometer readings
MOD-04 → MOD-05	NAV	slam_state_t	20 Hz	Robot pose (x, y, θ) and ICP confidence
MOD-04 → MOD-05	NAV	occ_map_t	On map update	Occupancy grid pointer; 0=free, 100=occupied
MOD-05 → MOD-04	NAV	checkpoint_req_t	On junction arrival	CP_REQ_SNAP request with checkpoint_id
MOD-04 → MOD-05	NAV	checkpoint_ack_t	On checkpoint response	Snapped pose or CP_ACK_FAIL
MOD-04 → MOD-02	Digital Twin	telemetry_t	5 Hz	Pose snapshot for WebSocket visualization
MOD-06 → MOD-04	Sensors	sensor_reading_t	10 Hz	Auxiliary proximity data for map enrichment

5.5 Return Codes

Code	Value	Meaning
SLAM_OK	0	Operation completed successfully
SLAM_ERR_INIT	-1	Initialisation failure (e.g. LiDAR not found)
SLAM_ERR_LIDAR	-2	LiDAR communication or scan error
SLAM_WARN_LOW_CONF	1	ICP confidence below SLAM_CHECKPOINT_THRESH (non-fatal)

6. MOD-05 — NAV (Path Planning & Control)

6.1 What This Module Does

MOD-05 is the brain of autonomous movement. It takes `nav_goal_t` goals issued by MOD-01 (voice commands) and, using the occupancy grid and pose received from MOD-04, runs an A* planner to compute a collision-free waypoint sequence. A proportional heading controller ($K_p = 1.2$) then executes path-following at 50 Hz, converting heading error into differential left/right PWM commands sent to MOD-03.

MOD-05 also manages the navigation state machine: IDLE → PLANNING → FOLLOWING → AVOIDING → RECOVERING → GOAL_REACHED. When MOD-06 reports an obstacle within 0.50 m, MOD-05 halts the robot (WAITING) and waits up to ~10–15 seconds for the obstacle to clear. If it persists, MOD-05 transitions to REROUTING, selects the clearer lateral direction using MOD-06 detour advice, executes a lateral detour, and requests a checkpoint snap from MOD-04 to re-anchor the pose.

On every FSM state transition, MOD-05 publishes a `nav_status_t` to MOD-01 (for TTS announcement) and to MOD-02 (for the Digital Twin overlay).

6.2 Navigation FSM States

State	Value	Trigger / Condition	Action
IDLE	0	No active goal	Robot stopped; waiting for <code>nav_goal_t</code> from MOD-01
PLANNING	1	<code>nav_goal_t</code> received	A* path computation from current pose to destination
FOLLOWING	2	Path computed successfully	Proportional heading controller drives toward next waypoint
AVOIDING	3	<code>obstacle_alert_t</code> active < 0.50 m	Robot halts; starts 10–15 s obstacle timer
STUCK	4	No progress for <code>NAV_STUCK_TIMEOUT_MS</code> (3 s)	Triggers recovery behaviour (back + rotate)
RECOVERING	5	After stuck / avoidance timeout	Lateral detour + checkpoint pose correction
GOAL_REACHED	6	Within <code>NAV_GOAL_XY_TOL_M</code> (0.05 m) of final WP	Announces arrival via MOD-01 TTS; returns to IDLE
ABORTED	7	Unrecoverable error or <code>nav_cancel()</code>	Emergency stop sent to MOD-03

6.3 Public API

Function	Signature	Returns	Description
nav_init	nav_init(const nav_config_t *cfg)	nav_result_t	Initialises buffers, subscribes to MOD-04/03/06 data. Pass NULL for default parameters.
nav_deinit	nav_deinit(void)	void	Cancels active goal, sends zero-PWM to MOD-03, frees buffers.
nav_goto	nav_goto(const nav_waypoint_t *wp)	nav_result_t	Commands the robot to a single waypoint. Cancels any active mission first.
nav_goto_multi	nav_goto_multi(const nav_waypoint_t *wps, uint8_t count)	nav_result_t	Enqueues an ordered list of waypoints (up to NAV_MAX_WAYPOINTS = 64).
nav_cancel	nav_cancel(void)	nav_result_t	Cancels the current goal, sends zero-PWM, transitions to IDLE.
nav_pause	nav_pause(void)	nav_result_t	Decelerates to stop; retains the planned path for nav_resume().
nav_resume	nav_resume(void)	nav_result_t	Continues from the closest point on the retained path.
nav_replan	nav_replan(void)	nav_result_t	Forces A* re-planning from the current pose (e.g. after a map update).
nav_get_state	nav_get_state(void)	nav_runtime_state_t	Returns the current FSM state (no blocking).
nav_get_progress	nav_get_progress(nav_progress_t *out)	nav_result_t	Fills state, waypoints_total/done, dist_to_goal, path_length.
nav_register_cmd_vel_cb	nav_register_cmd_vel_cb(nav_cmd_vel_cb_t cb)	nav_result_t	Registers a 50 Hz callback to receive velocity commands (used by MOD-03).
nav_register_state_cb	nav_register_state_cb(nav_state_cb_t cb)	nav_result_t	Registers a callback for FSM state transitions (used by MOD-01 and MOD-02).

6.4 Key Constants

Constant	Value	Meaning
NAV_MAX_WAYPOINTS	64	Maximum waypoints per mission
NAV_PLANNER_MAX_NODES	8 192	A* node expansion limit per planning call
NAV_LOOKAHEAD_M	0.35 m	Pure-pursuit look-ahead distance
NAV_GOAL_XY_TOL_M	0.05 m	Position goal-reached tolerance
NAV_OBSTACLE_MARGIN_M	0.15 m	Safety inflation around robot footprint for planner
NAV_STUCK_TIMEOUT_MS	3 000 ms	No-progress timer before stuck detection triggers
NAV_CONTROL_PERIOD_MS	50 ms	Controller loop period (matches MOD-03 rate)
NAV_GOAL_YAW_TOL_RAD	0.10f	Goal heading tolerance (rad)
NAV_MIN_CLEARANCE_M	0.20f	Minimum clearance for a valid path (m)

6.5 Connections to Other Modules

Direction	Partner	Data Type	Rate	Notes
MOD-01 → MOD-05	Voice	nav_goal_t	On voice command	Destination name + waypoint_id
MOD-05 → MOD-01	Voice	nav_status_t	On FSM state change	Turkish TTS message
MOD-04 → MOD-05	SLAM	slam_state_t	20 Hz	Robot pose (x, y, θ) and confidence
MOD-04 → MOD-05	SLAM	occ_map_t	On map update	Grid for A* planning
MOD-05 → MOD-03	Hardware	motion_cmd_t	50 Hz	Left/right PWM + stop flag
MOD-03 → MOD-05	Hardware	motion_status_t	50 Hz	Actual speeds and stall flag
MOD-06 → MOD-05	Sensors	obstacle_alert_t	20 Hz	Triggers FOLLOWING → AVOIDING
MOD-05 → MOD-04	SLAM	checkpoint_req_t	On junction arrival	Pose-snap request
MOD-04 → MOD-05	SLAM	checkpoint_ack_t	On checkpoint response	Corrected pose
MOD-05 → MOD-02	Digital Twin	nav_status_t	On FSM state change	WebSocket overlay

7. MOD-06 — Sensor Integration

7.1 What This Module Does

MOD-06 manages all onboard proximity sensing beyond the primary LiDAR. It monitors the forward $\pm 25^\circ$ arc of the RPLidar A2 scan for objects closer than 0.50 m, and also reads two HC-SR04 ultrasonic sensors and two digital IR obstacle sensors connected via GPIO. Raw readings are passed through rolling-average noise filters to reject spurious spikes. Every 50 ms (20 Hz) the module consolidates all active readings into a single `obstacle_alert_t` struct that it provides to MOD-05.

When MOD-05 needs to reroute, it calls `sensor_get_best_detour()` to obtain the recommended lateral heading offset (e.g. -45° left or $+45^\circ$ right) based on whichever peripheral scan arc has more free space.

Raw `sensor_reading_t` snapshots are also forwarded to MOD-04 (for map enrichment) and MOD-02 (for visualization), at lower rates.

7.2 Sensor Configuration

Sensor	Type	Quantity	Forward Arc / GPIO	Update Rate
RPLidar A2 (forward arc)	LiDAR slice	1 (owned by MOD-04)	$\pm 25^\circ$ of 360° scan	20 Hz (from MOD-04 scan)
HC-SR04 Ultrasonic	Proximity	2	Front-left and front-right	20 Hz
IR Obstacle Sensor	Digital proximity	2	Front-left and front-right	20 Hz

7.3 Public API

Function	Signature	Returns	Description
<code>sensor_init</code>	<code>sensor_init(void)</code>	<code>sensor_status_t</code>	Configures GPIO pins for ultrasonic/IR sensors and sets up the LiDAR forward-arc data pipeline.
<code>sensor_update</code>	<code>sensor_update(void)</code>	<code>sensor_status_t</code>	Reads all sensors and runs noise filters. Call in the main control loop at <code>SENSOR_UPDATE_HZ</code> (20 Hz).
<code>sensor_get_alert</code>	<code>sensor_get_alert(obstacle_alert_t *alert)</code>	<code>sensor_status_t</code>	Fills the caller's <code>obstacle_alert_t</code> with the consolidated obstacle state (active, min_distance_m, bearing_deg).
<code>sensor_get_raw_reading</code>	<code>sensor_get_raw_reading(uint8_t id, sensor_reading_t *reading)</code>	<code>sensor_status_t</code>	Returns the raw distance reading from a specific sensor (0=front-left ultrasonic, etc.).
<code>sensor_get_best_detour</code>	<code>sensor_get_best_detour(float *best_bearing_deg)</code>	<code>sensor_status_t</code>	Evaluates left/right peripheral scans and returns the recommended heading offset (e.g. $\pm 45^\circ$).

7.4 Key Constants

Constant	Value	Meaning
SENSOR_MAX_RANGE_M	4.0 m	Maximum reliable sensor range for reactive avoidance
SENSOR_OBSTACLE_THRESH_M	0.50 m	Distance threshold to trigger an active obstacle alert
SENSOR_FORWARD_ARC_DEG	25.0°	Half-angle of the monitored forward arc ($\pm 25^\circ$)
SENSOR_UPDATE_HZ	20 Hz	Sensor read and alert publication rate
SENSOR_VERSION	"0.2"	Interface version string

7.5 Connections to Other Modules

Direction	Partner	Data Type	Rate	Notes
MOD-06 → MOD-05	NAV	obstacle_alert_t	20 Hz	Triggers MOVING → WAITING when active
MOD-06 → MOD-04	SLAM	sensor_reading_t	10 Hz	Raw readings for map enrichment
MOD-06 → MOD-02	Digital Twin	sensor_reading_t	5 Hz	Raw snapshot for visualization overlay

7.6 Return Codes

Code	Value	Meaning
SENSOR_OK	0	Success
SENSOR_ERR_INIT	-1	Hardware initialisation failure (GPIO or pipeline)
SENSOR_ERR_TIMEOUT	-2	Sensor failed to respond within its deadline
SENSOR_WARN_NOISE	1	High variance in readings detected (non-fatal warning)

8. Complete Inter-Module Connection Summary

The table below consolidates every data flow in the system. It can be read as a flat interface contract between all six modules.

#	From	To	Struct	Rate	Description
01	MOD-01	MOD-05	nav_goal_t	On voice cmd	Parsed destination name and waypoint_id from STT
02	MOD-05	MOD-01	nav_status_t	On FSM change	Navigation state + Turkish TTS string for speaker
03	MOD-03	MOD-04	odometry_data_t	50 Hz	Left/right encoder ticks and computed wheel speeds
04	MOD-03	MOD-04	imu_data_t	100 Hz	Gyro yaw-rate, accelerometer XYZ, and temperature
05	MOD-05	MOD-03	motion_cmd_t	50 Hz	Left/right PWM duty cycles and emergency stop flag
06	MOD-03	MOD-05	motion_status_t	50 Hz	Actual wheel speeds and motor stall detection flag
07	MOD-04	MOD-05	slam_state_t	20 Hz	Robot pose (x, y, θ) and ICP confidence score
08	MOD-04	MOD-05	occ_map_t	On map update	Row-major occupancy grid pointer + resolution info
09	MOD-05	MOD-04	checkpoint_req_t	On junction	Pose-snap request with checkpoint ID
10	MOD-04	MOD-05	checkpoint_ack_t	On CP response	Snapped pose or CP_ACK_FAIL result
11	MOD-06	MOD-05	obstacle_alert_t	20 Hz	Active flag, min distance, and bearing to obstacle
12	MOD-06	MOD-04	sensor_reading_t	10 Hz	Raw ultrasonic/IR distance for map enrichment
13	MOD-04	MOD-02	telemetry_t	5–10 Hz	Combined pose + nav + battery for WebSocket stream
14	MOD-05	MOD-02	nav_status_t	On FSM change	Navigation status for Digital Twin overlay
15	MOD-06	MOD-02	sensor_reading_t	5 Hz	Sensor snapshot for proximity visualization